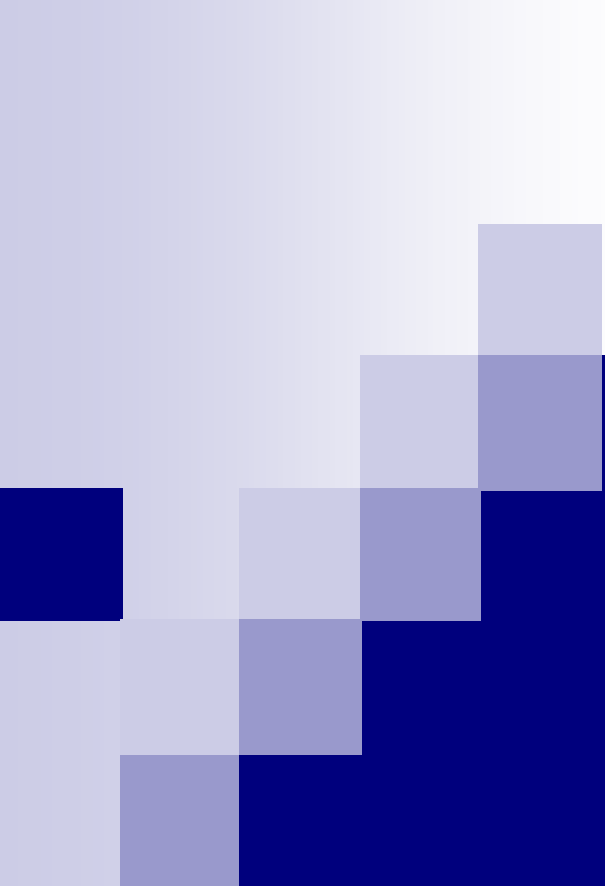


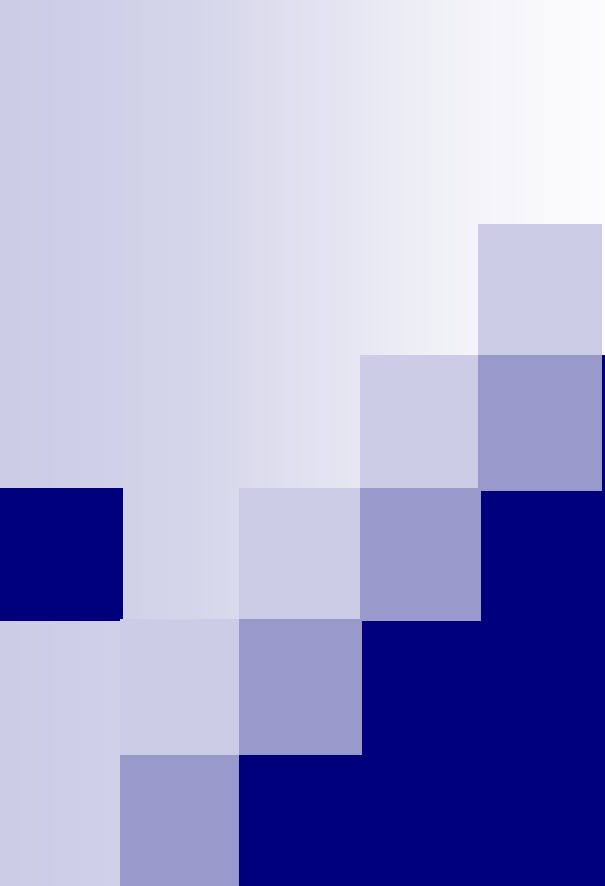


# Software Configuration Management – 4

Lecture # 36

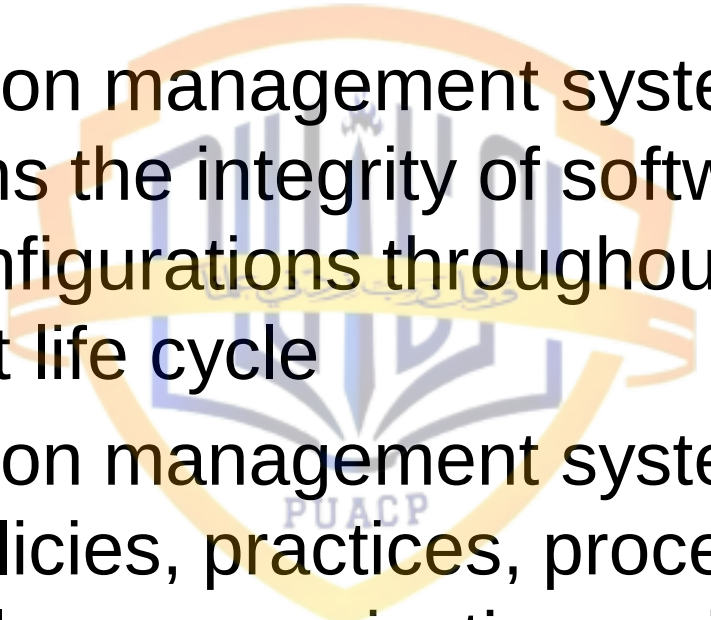
# Recap

- In the last lecture, we introduced practical aspects of software configuration management and discussed seven real-world considerations related to software configuration management
- Software configuration management is an integral part of software quality assurance program in any organization

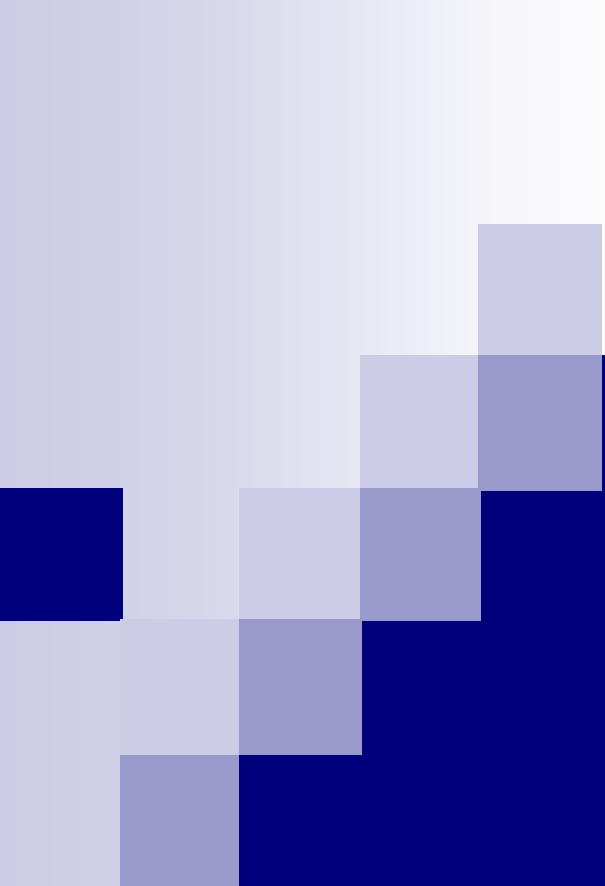


# Configuration Management Best Practices



- 
- 
- A configuration management system establishes and maintains the integrity of software artifacts and their configurations throughout the software development life cycle
  - A configuration management system includes the set of policies, practices, procedures, and tools that help an organization maintain its software

- 
- Today, we will discuss software configuration management best practices with respect to different aspects of software configuration management



# Practices for Managing Versions of Software Artifacts



# Practices for Managing Versions of Software Artifacts - 1

- All source artifacts should be under configuration control
- All artifacts used to produce an artifact of a delivery should be under configuration control
- Work within managed, private workspaces

# Practices for Managing Versions of Software Artifacts - 2

- Save artifacts at the completion of intermediate steps of a larger change
- Regularly synchronize development with the work of others
- Define policies of branches, codelines, and workspaces



# Practices for Managing Versions of Software Artifacts - 3

## □ Codelines

- Identify how many development codelines are available and name each one – typically one
- Identify how often development codelines must be integrated
- Identify who can make changes to a codeline, and under what circumstances
- Identify if parallel or non-parallel development is permitted for each codeline

# Practices for Managing Versions of Software Artifacts - 4

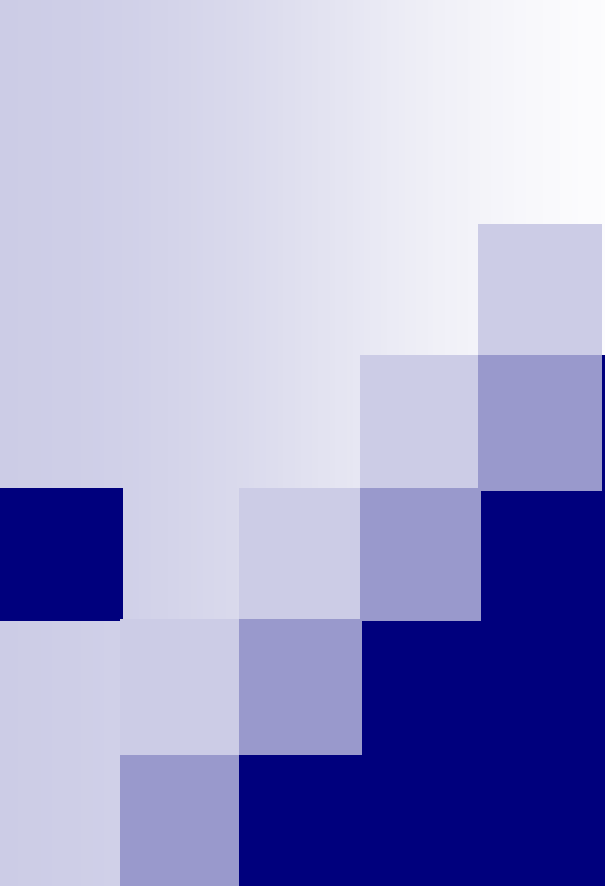
## □ Branches

- Identify the necessary conditions for creating a branch
- Identify the necessary conditions for merging a branch back into its source code line
- Identify the maximum period that an artifact can undergo change without being saved within the configuration management system

# Practices for Managing Versions of Software Artifacts - 5

## □ Workspaces

- Identify who can read and write artifacts of a workspace
- Identify who can add artifacts to a workspace or delete them from a workspace

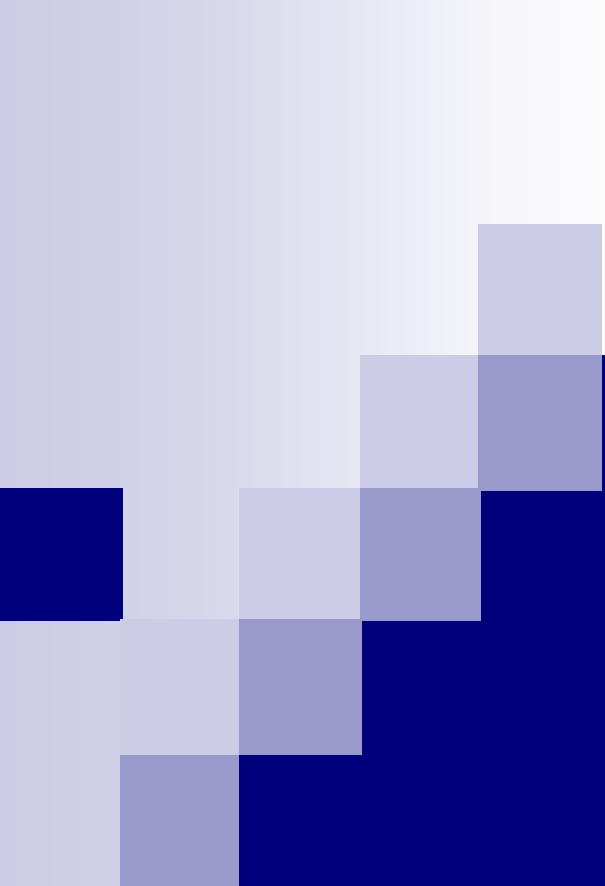


# Practices for Controlling Changes to Software Artifacts

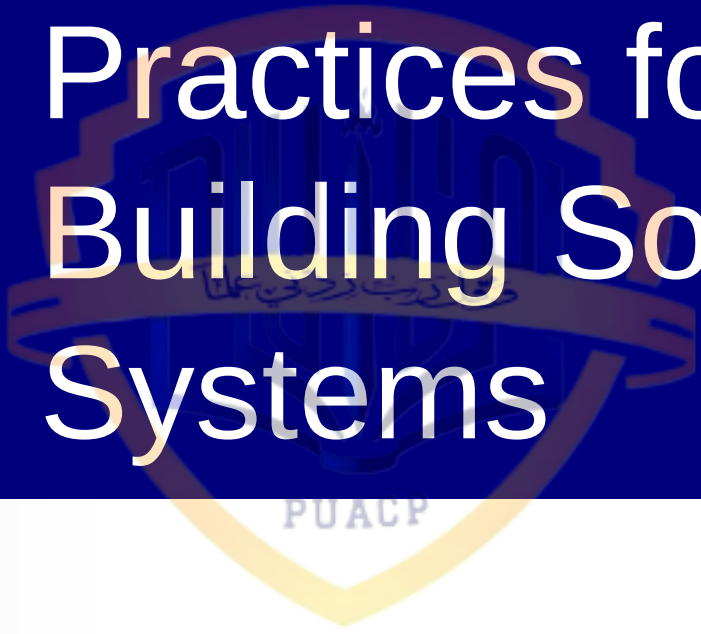


# Practices for Controlling Changes to Software Artifacts - 1

- Document identified software defects
- Create a defined process for requesting and approving changes
  - Use a change control board
  - Use change packages (aggregate collection of related changes)
- Apply defect repairs to existing releases and ongoing development efforts

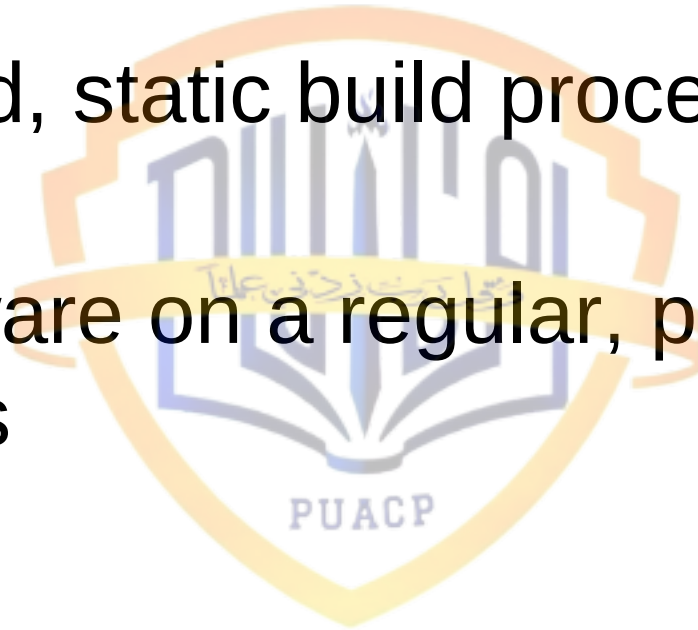


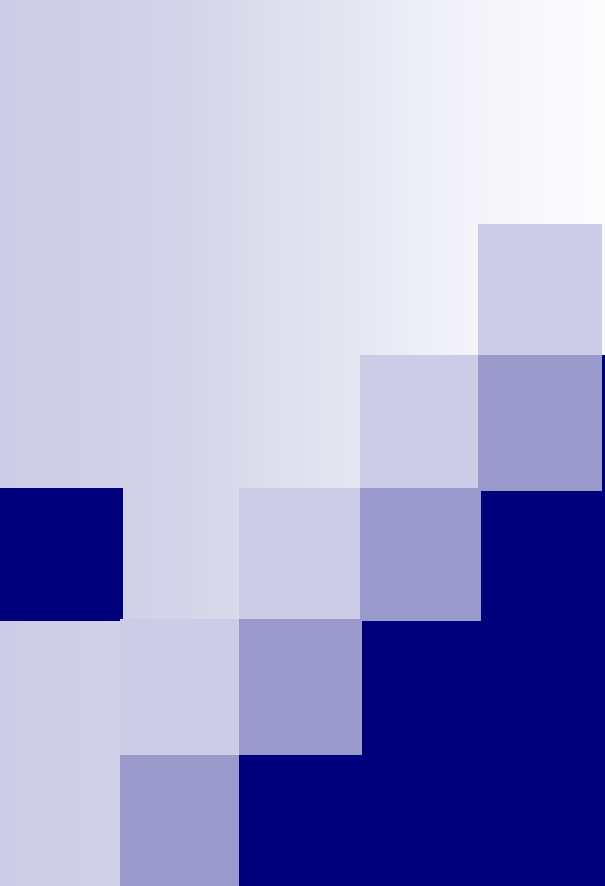
# Practices for Building Software Systems



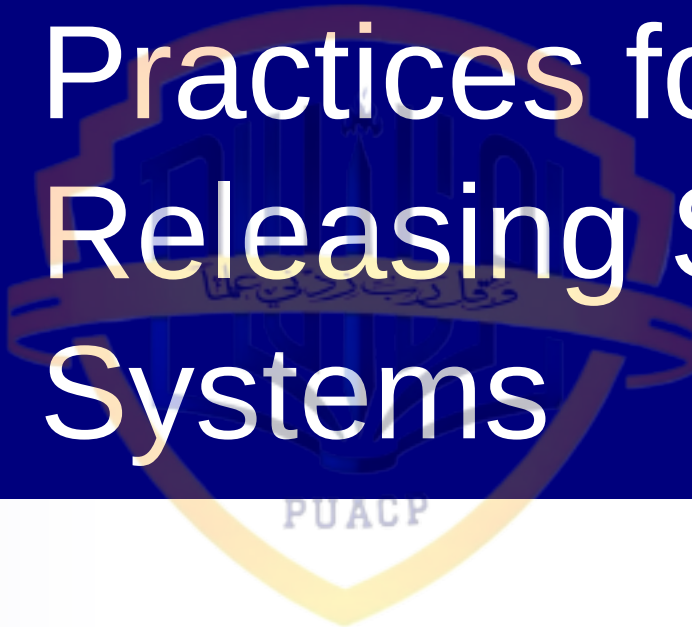
# Practices for Building Software Systems

- Use shared, static build processes and tools
- Build software on a regular, preferably daily, basis





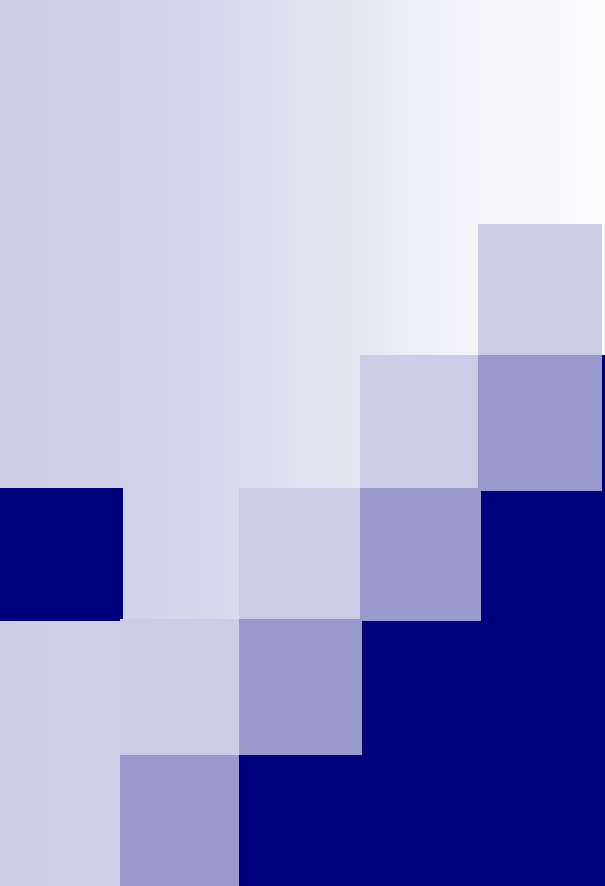
# Practices for Releasing Software Systems





# Practices for Releasing Software Systems

- Maintain a unique read-only copy of each release
- A version manifest should describe each software release
- Software artifacts that comprise a release should adhere to defined acceptance criteria
- Configuration management tools should provide release updates

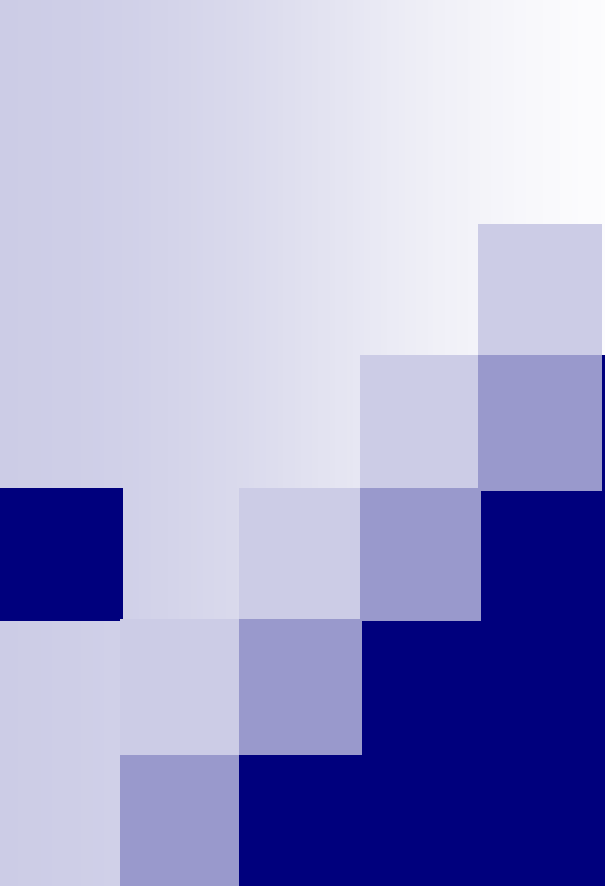


# Practices for Maintaining the Integrity of Software Artifacts



# Practices for Maintaining the Integrity of Software Artifacts

- Use a software tool to perform configuration management functions
- Repositories should exist on reliable physical storage elements
- Configuration management repositories should undergo periodic backups
- Test and confirm the backup process (backup and restore repositories)



# A Procedure for Creating a Configuration Management System



# Procedure for Creating a CMS - 1

- Acquire highly reliable and redundant physical storage and processing elements for the software repository
- Identify configuration management administrator, who is responsible for numerous tasks. The key tasks include
  - The creation of configuration management accounts and the assignment of capabilities to them

# Procedure for Creating a CMS - 2

- ☐ The enforcement of defined configuration management policies and procedures
- ☐ The building of internal and external deliveries
- Define a backup procedure to regularly back up configuration management repositories to nonvolatile storage and periodically purge them of redundant or useless data. This procedure should identify when incremental and full backups are done

# Procedure for Creating a CMS - 3

- Define a procedure that verifies that the backup process functions correctly
- Determine whether work must be authorized. If work must be authorized, then:
  - Establish a change control board
  - Assign people to the change control board
  - Define rules for approving changes to artifacts

# Procedure for Creating a CMS - 4

- Identify the number of development lines. Typically, one is sufficient. If more than one development line is needed, then:
  - Specify the frequency of the integration of each development line
- Identify the number of new tasks that an individual can work on simultaneously



# Procedure for Creating a CMS - 5

- Determine whether parallel development is permitted
- Determine if branches can be created for tasks other than parallel development or the development of releases. If branches can be created then:
  - Identify who can create the branches
  - Specify under what conditions the branches can be created
  - Establish the criteria for determining when merges are performed

# Procedure for Creating a CMS - 6

- Determine who can create workspaces
- Specify standard workspaces
- Identify what information should be specified for each new development task, change request, and anomaly report. Consider performing the following as required actions for each change request and anomaly report
  - Estimate the size of the change
  - Identify any alternative solutions
  - Identify the complexity of the change and the impact on other systems

# Procedure for Creating a CMS – 6-1

- Identify when the need exists
- Identify the effect the change will have on subsequent work
- Estimate the cost of the change
- Identify the criticality of the change request
- Identify if another change request will solve this problem
- Identify the effort to verify the change

# Procedure for Creating a CMS – 6-2

- Identify who will verify the change
- Identify whether the right people are available to work on the request
- Identify the impact on critical system resources, if this is an issue
- Identify the length of time that the change request has been pending

# Procedure for Creating a CMS - 7

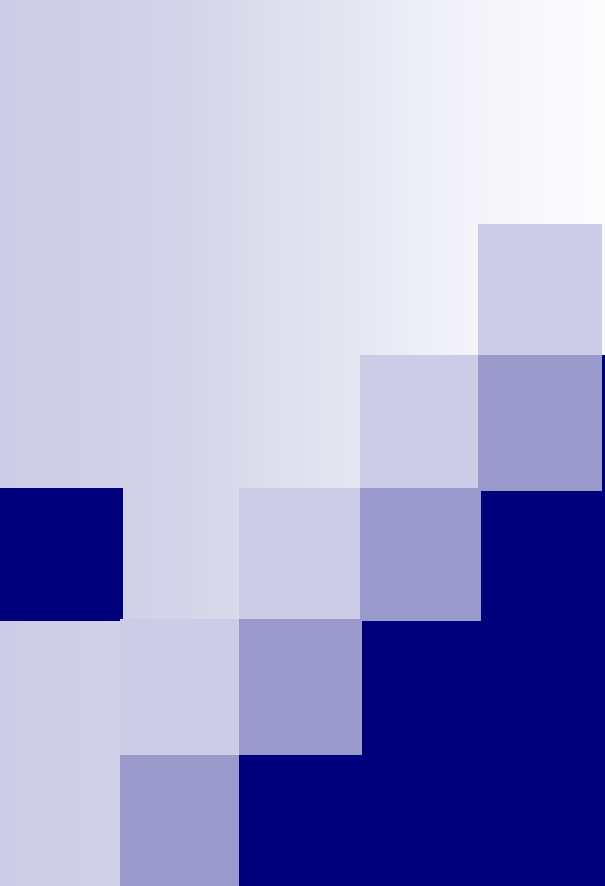
- Select metrics to gather
  - Number of change requests submitted
  - Number of change requests reviewed and approved for resolution
  - Number of change requests resolved and length of resolution
  - Number of anomaly reports submitted
  - Number of anomaly reports reviewed and approved for correction

# Procedure for Creating a CMS – 7-1

- Number of anomaly reports corrected and length of correction
- Number of artifacts changed
- Number of artifacts changed more than once (these should be characterized by the number of changes and frequency of the changes)

# Procedure for Creating a CMS - 8

- Acquire a configuration management tool that is able to manage software configurations, document identified software defects, and produce software releases
- Automate the policies, practices, and procedures as much as possible



# Best Change Control Practices Industry-Wise





# Best Change Control Practices Industry-Wise

- MIS software projects
- Outsourced software projects
- System software projects
- Commercial software projects
- Military software projects

# MIS Software Projects - 1

- Change Control Boards
  - For all projects larger than 5,000 function points
  - CCB usually has three to seven people
- Automated Change Control
  - For all deliverables, which include requirements, specifications, design documents, source code, test plans, user documentation, and training material
  - Package should flag recommended changes

# MIS Software Projects - 2

- Function Point Metrics for Changes
  - For projects larger than 15 function points in size, estimate and measure the function totals of all changes to software project
  - These changes include requirements creep, removal (or deferral) of feature, and requirements churn

# Outsourced Software Projects - 1

- Larger outsource vendors are often quite expert in implementing change control mechanisms
- Once an application has been deployed, new features and modifications average approximately 7% per year for several years in a row (i.e., new and changed features will approximate 7% of function points)

# Outsourced Software Projects - 2

- Change Estimation and Costing in Contracts
  - Specific clauses for change control are included in the outsource agreements
  - The forms of clauses vary with specific needs of the client, but are often based on the predicted volumes of the changes
  - Initial set of requirements have a fixed price, but new requirements will be included at a higher price

# Outsourced Software Projects - 3

- Function Point Metrics for Changes
  - Estimate and measure the function point totals of all changes to software projects larger than 15 function points
- Change Control Boards
  - For all projects larger than 5,000 function points
  - CCB usually has three to seven people

# Outsourced Software Projects - 4

- Automated Change Control
  - For all deliverables, which include requirements, specifications, design documents, source code, test plans, user documentation, and training material
  - Package should flag recommended changes
  - Automated change control tools that support only source code are adequate for projects larger than 100 function points

# Systems Software Projects - 1

- For these kinds of projects, changes during development can occur for a much wider variety of reasons than those found with internal information systems
- Systems software control physical devices



# Systems Software Projects - 2

- Change Control Boards
  - For all projects larger than 10,000 function points
  - CCB usually has three to seven people
    - Primary client
    - Project office
    - Development team
    - Hardware portion of the application

# Systems Software Projects - 3

- Automated Change Control
  - For all deliverables, which include requirements, specifications, design documents, source code, test plans, user documentation, and training material
  - Package should flag recommended changes
  - Automated change control tools that support only source code are adequate for projects larger than 100 function points

# Systems Software Projects - 4

- Function Point Metrics for Changes
  - Estimate and measure the function point totals of all changes to software projects
  - This data can be used for charge-backs and billing, and to ascertain monthly rate of requirements creep

# Systems Software Projects - 5

- Cost Estimates for Changes
  - Cost-estimating changes and cost measurement of changes are both difficult
  - Use automated estimation tools and function point metrics

# Systems Software Projects - 6

- Requirements Tracing and Changes
  - Each design feature and even each code module is traced back to specific requirement
  - Requirements tracking requires fairly sophisticated automation, and also demand a formal change control board

# Commercial Software Projects - 1

- Commercial software vendors may market the same application on different hardware platforms
- They may offer the same application in different national languages
- When major changes occur, they affect dozens of versions at the same time

# Commercial Software Projects - 2

- Change control is a key technology for commercial software vendors
- Automated Change Control



# Military Software Projects - 1

- The military software community was an early adopter of change control packages
- Change control starts during initial development and continues until an application is retired



# Military Software Projects - 2

- Change Control Boards
  - For all projects larger than 10,000 function points
  - CCB usually has three to seven people
    - Primary client
    - Project office
    - Development team
    - Hardware portion of the application for hybrid projects

# Military Software Projects - 3

- Automated Change Control
  - For all deliverables, which include requirements, specifications, design documents, source code, test plans, user documentation, and training material
  - Package should flag recommended changes
  - This is one of the 16 best practices identified by the Airlie Council

# Military Software Projects - 4

## ■ Function Point Metrics for Changes

- Estimate and measure the function point totals of all changes to military projects
- This data can be used to ascertain the monthly rate of requirements creep

## ■ Cost Estimates for Changes

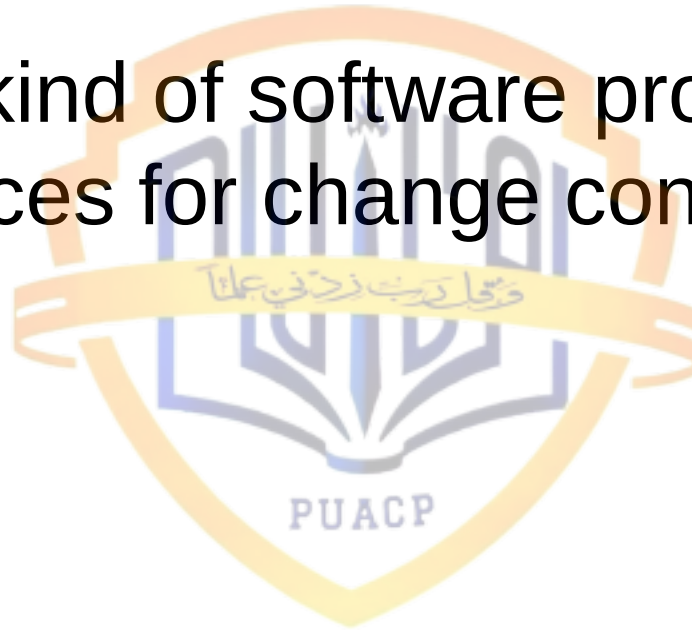
- Cost-estimating changes and cost measurement of changes are both difficult
- Use automated estimation tools and function point metrics

# Military Software Projects - 5

- Requirements Tracing and Changes
  - Each design feature and even each code module is traced back to specific requirement
  - Requirements tracking requires fairly sophisticated automation, and also demand a formal change control board
  - Change control in the military domain is not limited only to software changes

# Summary

- For every kind of software project, employ best practices for change control



# References

- Software Engineering Quality Practices by Ronald K. Kandt, Auerbach Publications, 2006 (Chapter 5)
- Software Assessments, Benchmarks, and Best Practices by Capers Jones, AW 2000 (Chapters nn)